Copyright 2026 David Gerald Morey

If you find this information useful, please add me on linked in as I'm an independent, se lf taught dev and could use the connections.

NOTE: These notes are added to as I figure out how to use the PIC64GX curiosity kit, and as such are likely to be incomplete. For any errors or updates feel free to message me, a s I would appreciate it immensely. I will continue to update them as I get more acquainte d with the pIC64GX RISC-V architecture, which I am planning on running TensorFlow models bare metal on and passing back to Linux for heavier processing via OpenCV.

https://www.linkedin.com/in/dave-morey-protovelop

List of Requirements:

VS Code
VS Code Pic64GX SDK extension
VS Code Serial Monitor extension (not required but highly recommended)
Python v 3.8 - 3.12 (3.12 recommended, DO NOT USE HIGHER VERSION), recommend to use a ven v in the project directory for this version
CMake
WSL 2

Microchip also have a tool for converting TensorFlow models to package for bare metal on polarfire SoCs called VectorBlox (still figuring out how to use it - will provide a prope r section for it at a later date).

PIC64GX BUILD HOW TO

Note: Pic64gx requires python 3.12 - recommended to setup a venv for this python version will also need vscode and downloaded pic64gx sdk

set up drivers - for comms you will need to download either putty or vscode's serial moni tor extension. I prefer to have both as putty allows for easier copying and pasting of co de - particularly when using things like Linux's nano text writer - whereas vscode serial monitor will allow all three ports to be monitored at once (which will be needed for har t core debugging during AMP). WHICHEVER IS USED, MAKE SURE THE BAUD RATE, BITS AND PARITY ETC IS SET UP PROPERLY. Check the PIC64GX quickstart guide on setting up drivers.
NOTE there was a problem I was having where the ports were becoming locked after plugging a fender quantum lt2 device into the back computer port. In devices menu one of the comm ports - the one supposed to be located at serial converter D when checked in properties - had in brackets "Interface" as its name and its location in properties was some random 0.0.0.** number. The solution, I found, was to remove the fender quantum lt2 device, then remove the PIC64GX before plugging it back in, to which the comm port number then ch anged, its name no longer had "interface" in brackets and the location in properties chan ged back to serial converter D (currently comm11 but originally was comm8). After this, I had communication coming through vs code serial monitor channel connected to serial conv erter B (currently comm9). If the unplugging of devices doesn't work, try uninstalling th e driver for the corrupted - location 0.0.0 - driver first. I plugged the lt quantum 2 in after this process and i still had comms coming through comm9. Upon booting I had to pre ss esc command through comm9 as the pic logo was being loaded, upon which i got comms thr ough comm10 (connected to serial converter C) giving boot options into Linux. I would rec ommend having the curiosity kit plugged in whilst setting up the SDK, but i don't know if this really matters.

Write down those MCHP comm port numbers, as they'll be needed during debugging.

UNDERSTANDING THE ARCHITECTURE (needs fact checking):

PIC64GX is a 64 MPU with 5 different hart cores, one of which can run Linux - named hart 0 which is an e51 type of core - and 4 which can run c applications bare metal, and which are cores of type u54. The PIC can be run in the default Symmetric Multiprocessing (SMP) mode which uses identical processors for all tasks, which can be inefficient due to excessive resource allocation for certain workloads, or it can be configured for Asymmetric Multi Processing (AMP) operation, where various processing contexts are separated and optimized for specific tasks. SMP utilises a shared memory structure whereas AMP has memory partitions specifically for each core. In addition to this AMP can be configured to run a Linux operating system on one core and either a bare metal or RTOS application on the other cores. Bare metal configurations operate without an operating system directly interacting with hardware through memory mapped registers, and is usually suitable for low cost/low complexity applications where real time deadlines are not critical. Real Time Operating Systems (RTOS) on the other hand, can be configured for deterministic operation, where a guaranteed time frame for code execution and response to certain events is needed. RTOS systems are much complicated and introduce high memory usage as well as debugging challenges if not well understood.
The first thing to decide is which of the 3 operational configurations are needed, as this will determine how the SDK is used.

For the Hart 0 e51 core, a Linux image is needed. Canononical have a prebuilt Linux image for the PIC64GX, but it is a full scale headless operating system, meaning it has a lot of general purpose drivers and setups that that make booting up take quite some time. To build a custom Linux OS with only the needed drivers and packages that boots much quicker, either buildroot or Yocto need to be used. Buildroot is a lot easier but Yocto provides much more customization at a cost of complexity.
Then comes the problem of taking the c source code placed in the hart core folders of the project and actually burning into the PIC64GX's on board memory so it runs independently of a debugging environment connected to the host computer. The method by which this is done is to use Hart Software Services (HSS) and the xpack-riscv-none-elf-gcc-13.2.0-2-win32-x64 toolchain to boot payloads containing the hart configurations: Yocto/buildroot is used to create a uboot.bin file whilst the RISCV toolchain is what is used to create a payload yaml using both the {project_name}.elf and {project_name}.bin files created after building the project (look in the project's "build" folder). An ELF file is itself a machine binary file that contains your own compiled memory locations and code.
It requires both a .c source file and an .ld linker file to create - the c soure file should be what ever is in the {project_name} hart folders, whilst the linker should be found in the {project_name} build folders after building.
A linker (.ld) file tells the compiler where everything lives in memory; it tells the linker to "put this file at this memory address" look at examples to get an idea of how the linker file should be written.

typical linker file:

```
        MEMORY
    {
      RAM (rwx) : ORIGIN = 0x08000000, LENGTH = 64K
    }

    SECTIONS
    {
      . = 0x08000000;

      .text : {
        *(.text*)
      }

      .rodata : {
        *(.rodata*)
      }

      .data : {
        *(.data*)
      }
```

```
      .bss : {
        *(.bss*)
      }

      . = ALIGN(8);
      _stack_top = . + 0x1000;
    }
```

IMPORTANT: MAKE SURE PIC64GX SDK AND HART SOFTWARE SERVICES ARE INSTALLED AND ADDED TO PATH. Current installation is in c:/{subfolder}

1. VS Code should be installed along with the PIC64GX SDK and serial monitor extensions. Whilst not strictly needed, as alternatives like Putty can be used instead, the serial monitor extension is recommended as it gives the opportunity to monitor three different UART comm ports at once, so it can be set up to debug the com ports used by the various PIC64GX hart cores. Putty is also recommended as it makes editing Linux's nano txt editor easier with copy and pasted commands from a windows host pc, though is only useful after SSH has been set up, but will only give you one terminal instance.
The SDK on the other hand is non negotiable as it contains the drivers needed for these PIC64GX comm ports to become operational.
IMPORTANT: Ensure a decent high current USB type C cable is connected to the PIC64GX curiosity kit, and that the Type A side connects to a USB 3 port on a computer or laptop - USB hubs are not recommended as per Microchip's datasheet for the curiosity kit; failing to do so could cause headaches with trying to find the correct devices in the device manager during driver install. In Device manager look for other devices and set up as per the official microchip quick start guide found at https://ww1.microchip.com/downloads/aemDocuments/documents/MPU64/ProductDocuments/UserGuides/production-kit-qsguide/Curiosity-PIC64GX1000-Kit_QSGuide.pdf (when installing driers through device manager they can be found in the SDK instal folder - check the quickstart guide for location.

NOTE: pic64gx-hart-software-services has some baremetal examples in the core folders of the application folder which are useful for testing

2. The PIC64GX SDK uses a specific version of python (between 3.8 - 3.12) upon initialisation to create the necessary files and folders for the workspace. Version 3.13 and above cannot (currently as at 10/09/26) be used due to a known dependency conflict. Rather than uninstalling any higher versions, which may cause dependency problems for other applications, a virtual environment (venv) can be setup so that the project only uses version 3.12, whilst other projects can still use version 3.13 and above where necessary.

2a. If not already done so, install python version 3.12 to c: drive from https://www.python.org/downloads/

2b. Create a python version 3.12 virtual environment, by opening CMD, and cd to beside the sdk folder (currently C:\\Users\\{user}\\Desktop\\My_Projects\\PIC64GX_Projects). run these commands in powershell:

```
    py -3.12 -m venv {venv\_name}
```

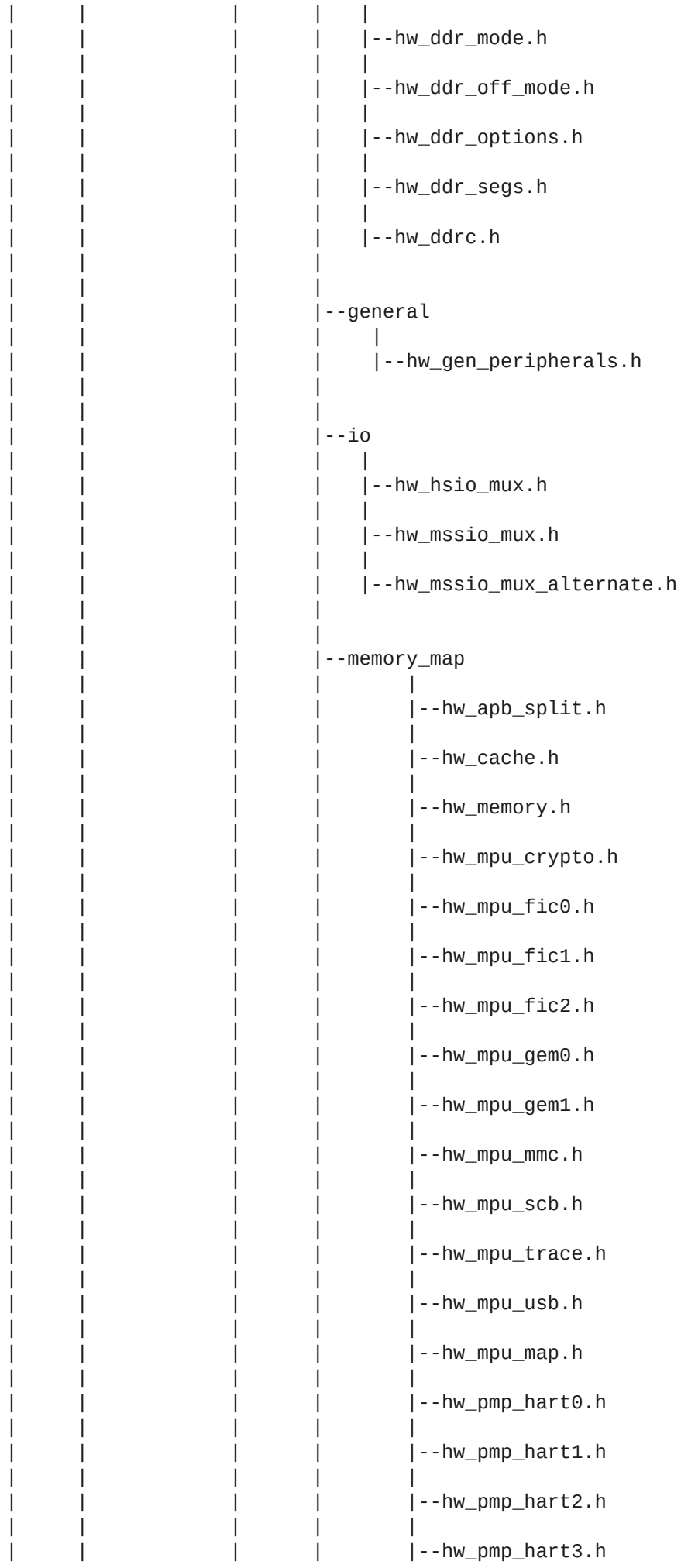
2c. then:

```
    {path to sdk}\\{venv\_name}\\Scripts\\activate.bat
```

2d. Point the project path to use the python 3.12 venv in environment variables. Then open vs code, press ctrl+shift+p and "python: select interpreter: then choose the one at the venv location you just created; this will tell vscode you want your project to use the version 3.12 virtual environment as the default python application for open .py files, circumnavigating any higher versions that might be installed to c: drive. A word of CAUTION: Vscode has a tendency to automatically select the wrong venv when opening a project (atleast, it has for me) - make sure the proper venv is showing at the bottom right of the screen. If not, click on it and select from the 3.12 venv you just set up.

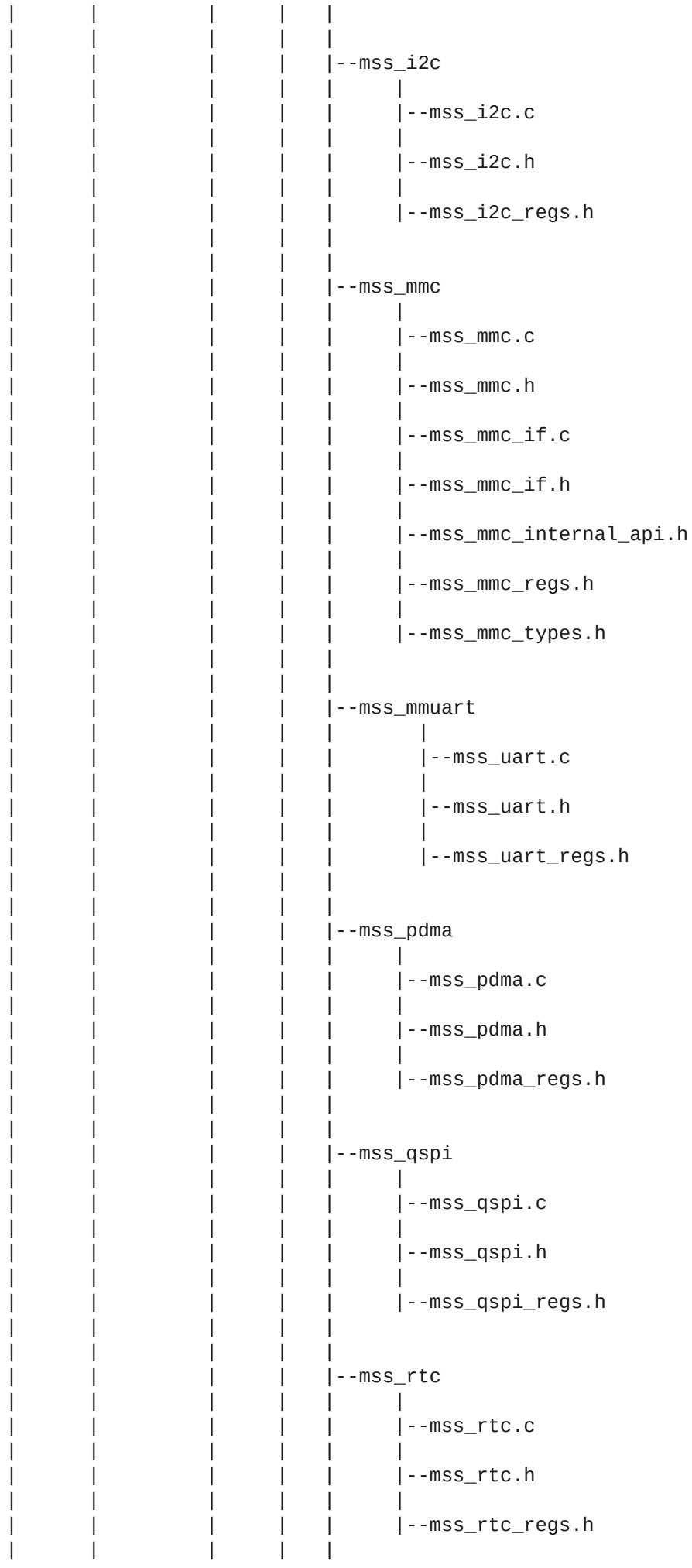
2e. Ensure pip (use powershell for these):

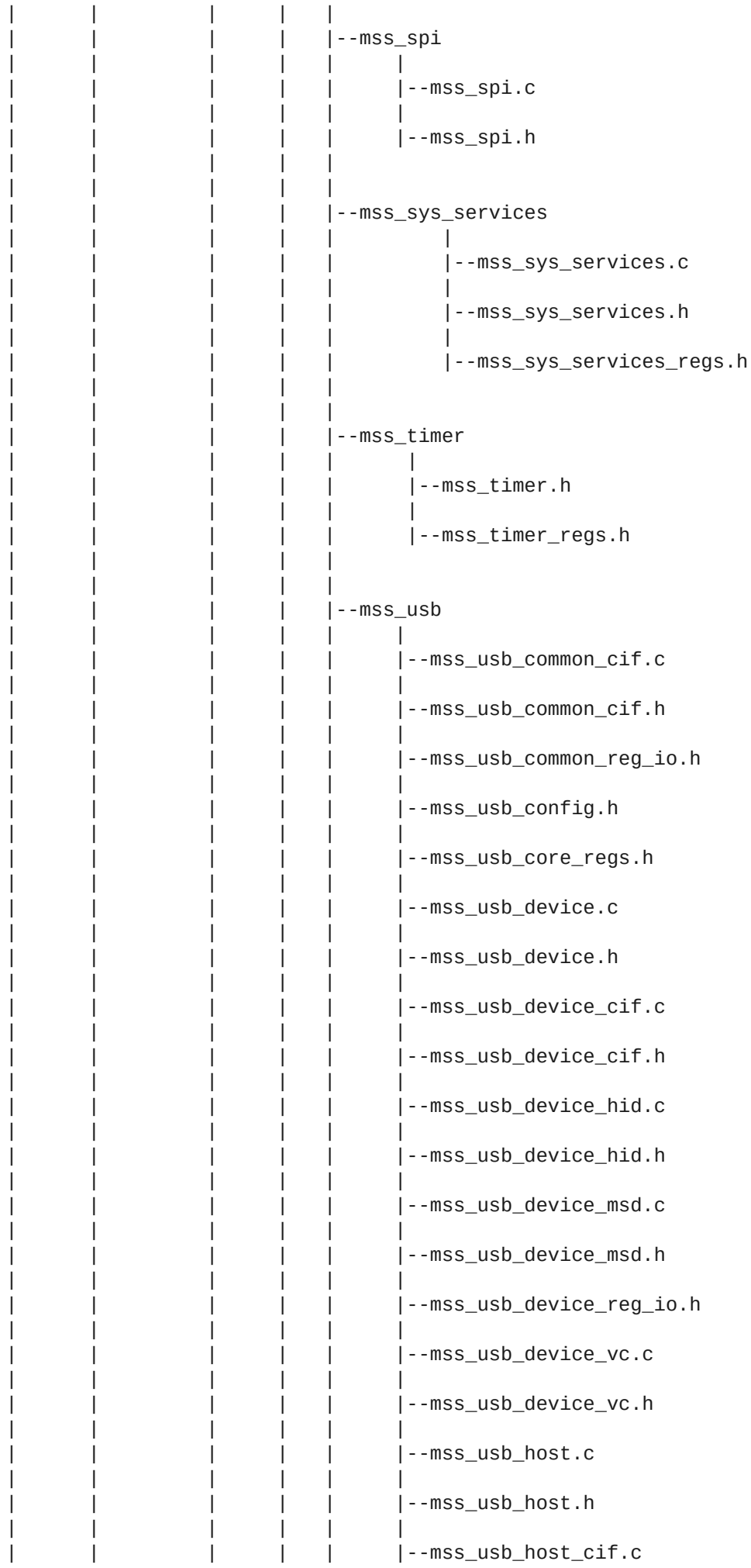
```
    ** this is my project location
    C:\\Users\\{user}\\Desktop\\My\_Projects\\PIC64GX\_Projects\\venv\_3.12\\Scripts\\pyt
hon.exe -m ensurepip --upgrade

2f. check pip version:

    ** this is my sdk install location
    C:\\pic64gx-sdk\\venv312\\Scripts\\python.exe -m pip --version

2g. install pip:

    C:\\pic64gx-sdk\\venv312\\Scripts\\python.exe -m pip install --upgrade pip

3. To set up the SDK properly, one should first locate the readme.md file in the pic64gx
sdk vscode extension folder (located at C:\Users\fende\.vscode\extensions\microchip.pic64
gx-sdk-extension-0.1.6). This readme has valuable information and instructional gifs on h
ow to initialise the sdk. NOTE that the sdk seems like it can be initialised in any folde
r, in which it will add the following folder structure to that folder (this may be incomp
lete due to writing it when I didn't realise I was having initialisation problems):

[Selected Folder]
    |
    |--MANIFEST            <<<<<< IMPORTANT: when opening a project from the SDK, you hav
e to select a folder containing this file
        |
    |--sdk
            |
            |--.vscode
            |     |
            |     |--c_cpp_properties.json
            |     |
            |     |--launch.json
            |     |
            |     |--settings.json
        |
            |
        |--boards
            |     |
            |     |--pic64gx-curiosity-kit
            |     |            |
            |     |            |--bsp_config
            |     |            |      |
            |     |            |      |--clocks
            |     |            |      |    |
            |     |            |      |    |--hw_clk_ddr_pll.h
            |     |            |      |    |
            |     |            |      |    |--hw_clk_msscfm.h
            |     |            |      |    |
            |     |            |      |    |--hw_clk_mss_pll.h
            |     |            |      |    |
            |     |            |      |    |--hw_clk_sgmii_cfm.h
            |     |            |      |    |
            |     |            |      |    |--hw_clk_sgmii_pll.h
            |     |            |      |    |
            |     |            |      |    |--hw_clk_sysreg.h
            |     |            |      |    |
            |     |            |      |    |--hw_mss_clks.h
            |     |            |      |
            |     |            |      |
            |     |            |      |--ddr
            |     |            |      |   |
            |     |            |      |   |--hw_ddr_io_bank.h
```


|--hw_ddr_mode.h
|--hw_ddr_off_mode.h
|--hw_ddr_options.h
|--hw_ddr_segs.h
|--hw_ddrc.h
|--general
|--hw_gen_peripherals.h
|--io
|--hw_hsio_mux.h
|--hw_mssio_mux.h
|--hw_mssio_mux_alternate.h
|--memory_map
|--hw_apb_split.h
|--hw_cache.h
|--hw_memory.h
|--hw_mpu_crypto.h
|--hw_mpu_fic0.h
|--hw_mpu_fic1.h
|--hw_mpu_fic2.h
|--hw_mpu_gem0.h
|--hw_mpu_gem1.h
|--hw_mpu_mmc.h
|--hw_mpu_scb.h
|--hw_mpu_trace.h
|--hw_mpu_usb.h
|--hw_mpu_map.h
|--hw_pmp_hart0.h
|--hw_pmp_hart1.h
|--hw_pmp_hart2.h
|--hw_pmp_hart3.h

```
|     |            |      |       |
|     |            |      |       |--hw_pmp_hart4.h
|     |            |      |
|     |            |      |
|     |            |      |--sgmii
|     |            |      |    |
|     |            |      |    |--hw_sgmii_tip.h
|     |            |      |
|     |            |      |--bsp_config.h
|     |            |
|     |            |
|     |            |--drivers_config
|     |            |        |
|     |            |        |--coreplex
|     |            |              |
|     |            |              |--coreplex_ihc
|     |            |                      |
|     |            |                      |--coreplex_ihc_add_mapping.h
|     |            |                      |
|     |            |                      |--coreplex_ihc_config.h
|     |            |
|     |            |--pic64gx_hal_config
|     |                      |
|     |                      |--coreplex_sw_config.h
|     |                      |
|     |                      |--readme.tx
|     |
|     |--renode
|     |    |
|     |    |--platforms
|     |    |      |
|     |    |      |--cpus
|     |    |          |
|     |    |          |--pic64gx-curiosity.repl
|     |    |
|     |    |--scripts
|     |           |
|     |           |--macros.resc
|     |           |
|     |           |--pic64gx-curiosity.resc
|     |           |
|     |           |--pic64gx-curiosity-platform.resc
|     |
|     |--Kconfig
|
|--cmake
|    |
|    |--common.cmake
|    |
|    |--cross-compiler-gcc-tc.cmake
|    |
|    |--defaultPreset.json
|    |
|    |--preload.cmake
|    |
|    |--README.md
|
|--images
|    |
|    |--add-breakpoint.gif
|    |
|    |--breakpoint-reached.png
|    |
```

```
|    |--build-project.gif
|    |
|    |--create-project.gif
|    |
|    |--init-finished.png
|    |
|    |--init-new.gif
|    |
|    |--start-debugging.gif
|
|--platforms
|       |
|       |--mss
|       |   |
|       |   |--platform
|       |          |
|       |          |--drivers
|       |          |      |
|       |          |      |--mss
|       |          |      |    |
|       |          |      |    |--mss_can
|       |          |      |    |      |
|       |          |      |    |      |--mss_can.c
|       |          |      |    |      |
|       |          |      |    |      |--mss_can.h
|       |          |      |    |
|       |          |      |    |
|       |          |      |    |--mss_ethernet_mac
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_mac.c
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_mac.h
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_mac_regs.h
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_mac_sw_cfg.h
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_mac_types.h
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_mac_user_config.h
|       |          |      |    |          |
|       |          |      |    |          |--mss_ethernet_registers.h
|       |          |      |    |          |
|       |          |      |    |          |--null_phy.c
|       |          |      |    |          |
|       |          |      |    |          |--phy.h
|       |          |      |    |          |
|       |          |      |    |          |--ti_dp83867_phy.c
|       |          |      |    |          |
|       |          |      |    |          |--vsc8541_phy.c
|       |          |      |    |          |
|       |          |      |    |          |--vsc8575_phy.c
|       |          |      |    |          |
|       |          |      |    |          |--vsc8575_support.c
|       |          |      |    |          |
|       |          |      |    |          |--vsc8662_phy.c
|       |          |      |    |
|       |          |      |    |
|       |          |      |    |--mss_gpio
|       |          |      |    |      |
|       |          |      |    |      |--mss_gpio.c
|       |          |      |    |      |
|       |          |      |    |      |--mss_gpio.h
```


|--mss_i2c
|--mss_i2c.c
|--mss_i2c.h
|--mss_i2c_regs.h
|--mss_mmc
|--mss_mmc.c
|--mss_mmc.h
|--mss_mmc_if.c
|--mss_mmc_if.h
|--mss_mmc_internal_api.h
|--mss_mmc_regs.h
|--mss_mmc_types.h
|--mss_mmuart
|--mss_uart.c
|--mss_uart.h
|--mss_uart_regs.h
|--mss_pdma
|--mss_pdma.c
|--mss_pdma.h
|--mss_pdma_regs.h
|--mss_qspi
|--mss_qspi.c
|--mss_qspi.h
|--mss_qspi_regs.h
|--mss_rtc
|--mss_rtc.c
|--mss_rtc.h
|--mss_rtc_regs.h


|--mss_spi
|--mss_spi.c
|--mss_spi.h
|--mss_sys_services
|--mss_sys_services.c
|--mss_sys_services.h
|--mss_sys_services_regs.h
|--mss_timer
|--mss_timer.h
|--mss_timer_regs.h
|--mss_usb
|--mss_usb_common_cif.c
|--mss_usb_common_cif.h
|--mss_usb_common_reg_io.h
|--mss_usb_config.h
|--mss_usb_core_regs.h
|--mss_usb_device.c
|--mss_usb_device.h
|--mss_usb_device_cif.c
|--mss_usb_device_cif.h
|--mss_usb_device_hid.c
|--mss_usb_device_hid.h
|--mss_usb_device_msd.c
|--mss_usb_device_msd.h
|--mss_usb_device_reg_io.h
|--mss_usb_device_vc.c
|--mss_usb_device_vc.h
|--mss_usb_host.c
|--mss_usb_host.h
|--mss_usb_host_cif.c

```
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_cif.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_hid.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_hid.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_msc.c
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_msc.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_host_reg_io.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_regs.h
|      |         |     |   |     |
|      |         |     |   |     |--mss_usb_std_def.h
|      |         |     |   |
|      |         |     |   |
|      |         |     |   |--mss_watchdog
|      |         |     |   |        |
|      |         |     |   |        |--mss_watchdog.c
|      |         |     |   |        |
|      |         |     |   |        |--mss_watchdog.h
|      |         |     |   |
|      |         |     |   |
|      |         |     |   |--pf_pcie
|      |         |     |         |
|      |         |     |         |--pf_pcie.c
|      |         |     |         |
|      |         |     |         |--pf_pcie.h
|      |         |     |         |
|      |         |     |         |--pf_pcie_regs.h
|      |         |     |         |
|      |         |     |         |--pf_pcie_types.h
|      |         |     |
|      |         |     |--pf_pcie
|      |         |
|      |         |--hal
|      |         |
|      |         |--pic64gx_hal
|      |         |
|      |         |--CMakeLists.txt
|      |         |
|      |         |--Kconfig
|      |         |
|      |         |--LICENSE.md
|      |
|      |--CMakePresets.json
|
|--scripts
|
|--tools
|
|--gitignore
|
|--CMakeLists.txt
|
|--CMakePresets.json
|
|--Kconfig
|
|--README.md
```

4. To initialise the sdk, make sure vscode is opened at the folder you want the workspace (currently: C:\\Users\\{user_name}\\Desktop\\My_Projects\\PIC64GX_Projects) on my machine - this will be where the workspace is created!!! IMPORTANT: The venv 3.12 and its pip install must be setup and activated in this same folder PRIOR to initialising the SDK, as well as that folder being selected as the interpreter pathway in vscode after pressing Ctrl+shift+P. Go into environment variables and check that these paths are correct. Also close any other instances of the path location that might be open in file explorer to avoid access conflicts during initialisation.

Select "initialise sdk" in the bottom left blue/purple tool bar (will vary depending on machine/ vs code setup) of vscode, and select the project folder you just created (\[selected folder] in the tree above), or press ctrl+shift+P and search for "initialise sdk". This will clone necessary files from GitHub into that folder, including the MANIFEST file. There must be no errors on the vscode output window. if there is an error regarding certificates whilst downloading renode, open cmd and type the following:

Install/Update certifi:
    C:\Users\{user_name}\Desktop\My_Projects\PIC64GX_Projects\venv_3.12\Scripts\python.exe -m pip install --upgrade certifi

Run install certificates script:
    C:\Users\{user_name}\Desktop\My_Projects\PIC64GX_Projects\venv_3.12\Scripts\python.exe -m pip install pip-system-certs

re-run initialisation (or do this automatically in vscode by clicking on "initialise sdk")
    C:\Users\{user_name}\Desktop\My_Projects\PIC64GX_Projects\venv_3.12\Scripts\python.exe sdk/scripts/init.py

Create a new project by clicking on the PIC64GX SDK icon in the toolbar on the left side of the vscode screen, and then "Create SDK Project" in the panel that pops up (NOT the "create new project" button in "Open Configurator", as this is specifically for creating projects using AMP and will be configured below AFTER a global project has been created. Enter a project name and a project directory in the window that pops up. NOTE: I would not suggest manually trying to find a project folder; rather, TYPE in the same name as the project name or a name that is relevant to the project. In the bottom "Configure Project" window, "Boards Software Configuration", "Linker Script", "MSS Drivers", and "Platform" should automatically appear. Scroll down and click the "Create Project" button; a new folder will be created in the workspace folder alongside the sdk with the following structure:

```
    workspace folder (PIC64GX_Projects)
            |
            |--.vscode
            |
            |--boards
            |
            |--{new project folder}
            |        |
            |        |--temp (used for setup and auto deleted when not needed)
            |        |
            |        |--src
            |        |
            |        |--CMakeLists
            |        |
            |        |--proj.conf
            |
            |--modules
            |
            |--platforms
            |
            |--sdk
            |
            etc
```

Once the SDK has been initialised, selecting AMP and DDR from the explorer menu of vscode under the configurator dropdown should now open a window with selectable options rather than not doing anything. NOTE that Linux and Zephyr are the operating systems that can be chosen for each context with Zephyr being specifically an RTOS (Real Time Operating System) - more information on this RTOS is available from Microchip; the important thing to note is that one context can run Linux whilst the other runs Zephyr. AMP - Asynchronous Multi Processing - is the means by which they communicate with each other, as opposed to single context operation which is SMP (fact check this).
With AMP not selected, context A is automatically selected as processor, with context B being locked out of operation. Selecting AMP unlocks context B, but before a project can be created a few settings rules need to be implemented otherwise vscode will throw some errors:

First, atleast 1 hart core has to be dedicated to context B.

Secondly, context B also needs a main form of memory set up for it under the Memory Map/ DDR options of the side bar panel.

It is therefore recommend to use the following settings for AMP operation:

AMP selected
Context A = Linux
Context B = Zephyr
U54_1,2,3 = Context A
U54_4 = Context B
SPI1 = Context A
Inter-Hart Comms = Context A and B
Video pipeline = context A

All other options left as default (or change depending on needs)

IMPORTANT: In order to use click board plugged into the mikrobus socket, you need to ensure its SPI1 port has a context otherwise it will be disabled during build time. After the files have been generated in the project directory, check that its status is "okay" - not "disabled" - in the context.dtso file, mentioned below.

Once these settings have been chosen, pressing create project in the configurator window will generate a MANIFEST file in the selected project location. I would suggest selecting the project folder created before when creating a new project. Once this manifest file has been created, a message "Default DTS File:pic64gx-curiosity-kit-default.dts" and "Included DTSI File:pic64gx_default.dtsi" will appear above the open project/ create new project buttons of the configurator window. Pressing the "Generate" button will now generate the dtsi and dtb device tree files for AMP use. NOTE that the output will automatically choose the path to the installed HSS directory. I would recommend choosing a specific project folder that was created after sdk initialisation. If the HSS folder automatically appears in the top of the vscode window and doesn't seem to allow for a folder change, you will need to click the toolbar in the same area with the + symbol; after this, file explorer should automatically open when the generate button is clicked. Also NOTE that the configurator tab needs to be selected in the vscode window (treat it as if it is python file you wish to view) in order for the AMP/DDR settings to show.

If vscode is closed, or if the folder root selection is changed, you will need to REOPEN THE FOLDER WHERE THE MANIFEST FILE WAS CREATED - BEFORE YOU CAN ACCESS AMP AND DDR SETTINGS. PIC64GX Configurator - VSCode Extension has more information on this.

The following files will be generated, assuming amp is selected (if it isn't selected only the dtbo, dtso and overlay files for context A will be generated.

```
{selected output folder}
      |
          |--context_a.dtbo
          |
          |--context_a.dtso
          |
```

```
          |--context_a.overlay
          |
          |--context_b.dtbo
          |
          |--context_b.dtso
          |
          |--context_b.overlay
          |
          |--global.yaml
```

6\. The above files are used to configure the device for boot (refer to Mastering Embedded Linux Development book. page 54 onwards for instruction on how to use). Opening the global.yaml file will give the OS contexts as well as configured peripherals.

NOTE: the MANIFEST and global.yaml files are used to restore project settings whenever a project is opened. check PIC64GX Configurator - VSCode Extension file for more info.

Locate the context_a.dtso file (or whatever context spi1 was given) and replace the spi1status line with the following:

```
&spi1 {
 status = "okay";
   spidev@0 {
      #address-cells = <1>;
      #size-cells = <1>;
      compatible = "microchip,pic64gx-spidev";
      spi-max-frequency = <20000000>;
      reg = <0>;
   };
};
```

BARE METAL APPLICATIONS
Each project should have the following structure, which is used to run c source and header files on each hart:

```
[Selected Folder]  (should be C:\Users\{user}\Desktop\My_Projects\PIC64GX_Projects\{project_name} if steps above were followed)
      demo
    |
    |--CMakeLists.txt
    |
    |--proj.config
        |
    |--src
            |
            |--application
               |
               |--hart0
          |    |
          |    |--e51.c
          |
               |--hart1
          |    |
          |    |--u54_1.c    <<<<<< demo will edit this file
               |
               |--hart2
          |    |
          |    |--u54_2.c
          |
               |--hart3
```

```
            |    |
            |    |--u54_3.c
            |
                |--hart4
            |    |
            |    |--u54_4.c
            |
                |--inc
                |
                |--common.h
```

Navigate to the hart0_e51.c file and ensure it has a basic hello world application. if no ne exists, use the following.

```
#include "pic64gx_hal/mss_hal.h"
#include "mss_mmuart/mss_uart.h"


const uint8_t g_message[] = "\r\n Hello, World!\r\n";

void e51(void)
{
    (void) mss_config_clk_rst(MSS_PERIPH_MMUART0, (uint8_t) 1, PERIPHERAL_ON);

    MSS_UART_init(&g_mss_uart0_lo,
    MSS_UART_115200_BAUD,
    MSS_UART_DATA_8_BITS | MSS_UART_NO_PARITY | MSS_UART_ONE_STOP_BIT);

    /* Message on uart0 */
    MSS_UART_polled_tx(&g_mss_uart0_lo, g_message, sizeof(g_message));

    while (1U)
    {

    }
}
```

Navigate to hart1/ u54_1.c file, select all and replace the txt with the following applic ation that will flash an led on and off on the PIC64GX curiosity kit (original file can b e found at https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/64bit-mpu/ intro-to-pic64gx-development/):

```
    #include <stdint.h>
    #include "pic64gx_hal/mss_hal.h"
    #include "mss_gpio/mss_gpio.h"

    void delay(uint8_t seconds){
           for (uint64_t i = 0; i < (seconds * 11000000U); ++i) {
             __asm__("sll x0, x0, x0"); // Dummy instruction for delay
           }
    }

    void u54_1(void)
    {
            // Enable clock for GPIO1 peripheral
            (void)mss_config_clk_rst(MSS_PERIPH_GPIO1, (uint8_t) 1, PERIPHERAL_ON);

            // Initialize GPIO1 lower half
            MSS_GPIO_init(GPIO1_LO);

            // Configure pin 2 as output
            MSS_GPIO_config(GPIO1_LO, MSS_GPIO_2, MSS_GPIO_OUTPUT_MODE);
```

```
            // Blink loop
            while (1u)
            {
               delay(1);
               MSS_GPIO_set_output(GPIO1_LO, MSS_GPIO_2, 1u); // LED ON
               delay(1);
               MSS_GPIO_set_output(GPIO1_LO, MSS_GPIO_2, 0u); // LED OFF
            }
         }
```

NOTE: if we look at CMakeLists.txt and proj.conf contents, we get insight into how CMake created this file structure as well as the following configurations:

In CMakeLists.txt:

```
include(${WORKSPACE}/sdk/cmake/common.cmake)       =      pulls in Microchip's SDK build infrastructure

sdk_project(PIC64GX_Projects)         =     creates the SDK project

target_sources(${PROJECT_NAME} PUBLIC
    .../hart0/e51.c
    .../hart1/u54_1.c
    .../hart2/u54_2.c                 =    explicitly adds one source file for each hart
    .../hart3/u54_3.c
    .../hart4/u54_4.c
)
```

In proj.config:

```
CONFIG_PIC64GX_HAL_LAST_HART=4             =     corresponds to 5 hart setup

CONFIG_SGMII_SUPPORT=y
CONFIG_DDR_SUPPORT=y
CONFIG_MSSIO_SUPPORT=y                 =     generated project configured for the MSS platform and MMUART amongst other things
CONFIG_PLATFORM_MSS=y
CONFIG_PLATFORM=y
CONFIG_MSS_MMUART=y


(these lines may not be there due to discrepancy with set up. left in for the sake of learning how to implement memory management)
CONFIG_MEMORY="lim"
CONFIG_APPLICATION_STACK_SIZE=8192
CONFIG_APPLICATION_HEAP_SIZE=8192        =     application has been configured with specific memory resources rather than us having to invent them case
CONFIG_APPLICATION_RAM_SIZE=32768
```

We need to append CONFIG_MSS_GPIO=y to the bottom line of the proj.conf file, and save it:

```
    ...
CONFIG_PLATFORM=y
CONFIG_MSS_MMUART=y
CONFIG_MSS_GPIO=y
```

Now in vscode, make sure serial monitor extension has been downloaded and installed. Click on serial monitor tab in the output window at the bottom of the screen, and click the blue "+Open Additional Monitor" button 2 times until 3 serial monitors have been opened (sometimes one of them will become hidden and will need to be found by clicking either the serial monitor(#) text or expanding the window if it was minimised). Select the com ports associated with the MCHP-Debug-Ports in Device Manager>>>>Ports (COM and LPT) (Currently, COM9, COM10 and COM11 on my machine). Ensure Baud rate and other settings are correct by clicking on the cog icon for each monitor if they aren't displayed (the com port settings can be found by right clicking on com port one in Device Manger, Properties and the Port Settings tab). Click "Start Monitoring" for each of the three monitors.

BUILDING

6.The next step is to run build in the pic64gx plugin in the vscode window. For a build, the correct board needs to be selected from the SDK - currently: C:\Users\{user}\Desktop\My_Projects\PIC64GX_Projects\boards\pic64gx-curiosity-kit. NOTE that using the one in Hart software Services will result in a failed build. A project folder also needs to be selected; choose one that was created with the "Create SDK project" button after the SDK was initialised.

A build folder will be created in the workspace with the following structure:

```
    workspace folder (PIC64GX_Projects)
            |
            |--.vscode
            |
            |--boards
            |
            |--build
            |    |
            |    |--board
            |    |
            |    |--CMakeFiles
            |    |
            |    |--.config
            |    |
            |    |--.ninja_deps
            |    |
            |    |--.ninja_log
            |    |
            |    |--autoconf.h
            |    |
            |    |--build.ninja
            |    |
            |    |--cmake_install.cmake
            |    |
            |    |--CMakeCache.txt
            |    |
            |    |--compile_commands.json
            |    |
            |    |--{project_name}.bin
            |    |
            |    |--{project_name}.elf
            |    |
            |    |--{project_name}.lst
            |    |
            |    |--{project_name}-mine.map
            |    |
            |    |--generated-linker.ld
            |
```

```
            |--{new project folder}
            |        |
            |        |--temp (used for setup and auto deleted when not needed)
            |        |
            |        |--src
            |        |
            |        |--CMakeLists
            |        |
            |        |--proj.conf
            |
            |--modules
            |
            |--platforms
            |
            |--sdk
            |
            etc
```

RUNNING AND DEBUGGING

Once we have our {project_name}.elf file, we can run the application on the PIC64GX and debug it by clicking on the "Run and Debug" icon in the left side panel toolbar of vscode. This will open a new panel with a green play button at the top. Upon clicking this button a message saying "path to the elf file relative to the workspace etc build/*.elf (Press 'Enter' to confirm or 'Escape' to cancel)" and you will be prompted to enter the path into a text box. Type \build\{project_name}.elf and press enter.
When prompted for a debug probe, select PIC64GX. Debug Console should output a bunch of yellow text, and a toolbar with stop, and step over etc button should pop up at the top of vscode.

Switch the output window back to Serial Monitor and make sure that all three monitors are running. One of the com ports should out put Hello World, as the application source in the hart0/e51.c file dictates. An orange LED should start flashing on the PIC64GX curiosity board roughly every second, as dictated by the source in the hart1/u54_1.c file.